Operating Systems

Deadlocks

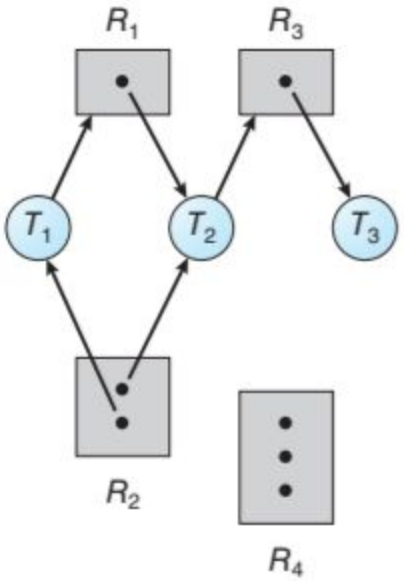
Deadlocks

- A system consists of a finite number of resources (physical or logical) to be distributed among a number of competing processes
- If a process requests an instance of a resource type, the allocation of any instance of the type should satisfy the request.
- Under the normal mode of operation, a process may utilize a resource in only the following sequence
  - **Request**. The process requests the resource.
  - **Use**. The process can operate on the resource
  - **Release**. The process releases the resource.
- The request and release of resources may be done using system calls
- A set of processes is in a deadlocked state when every process in the set is waiting for an event that can be caused only by another process in the set

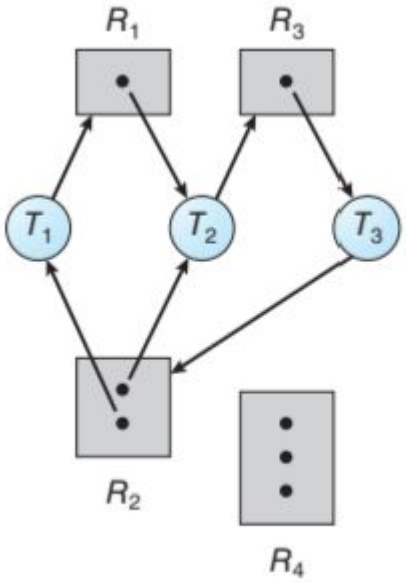
Deadlock Characterization

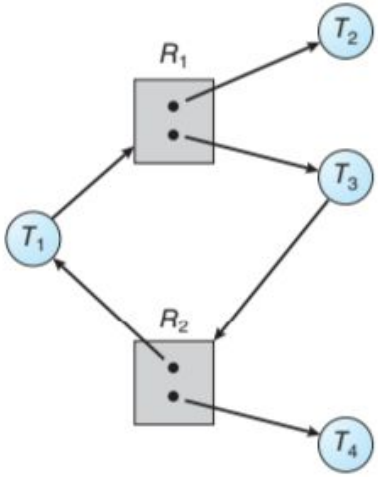
- **Mutual exclusion.** At least one resource must be held in a nonsharable mode; that is, only one process at a time can use the resource. If another process requests that resource, the requesting process must be delayed until the resource has been released.
- **Hold and wait.** A process must be holding at least one resource and waiting to acquire additional resources that are currently being held by other processes.
- **No preemption.** Resources cannot be preempted; that is, a resource can be released only voluntarily by the process holding it, after that process has completed its task.
- **Circular wait.** A set {P0, P1, ..., Pn} of waiting processes must exist such hat P0 is waiting for a resource held by P1, P1 is waiting for a resource held by P2, ..., Pn-1 is waiting for a resource held by Pn, and Pn is waiting for a resource held by P0.

Resource Allocation Graphs



Resource Allocation Graphs





Methods for Handling Deadlocks

- Prevent or Avoid Deadlocks
- Detect and Recover
- Ignore

Deadlock Prevention

Deadlock Prevention provides a set of methods to ensure that at least one of the necessary conditions for deadlock cannot hold

- **Mutual Exclusion**
- **Hold and Wait:**
    - Each process can request all its resources before it begins execution OR
    - A process should be allowed to request resources only when it has none
- **No Preemption:**
    - If a process is holding some resources and requests another resource that cannot be immediately allocated to it, then all resources the process is currently holding are preempted OR
    - If a process requests some resources, and they are allocated to some other process that is waiting for additional resources we can preempt the desired resources from the waiting process and allocate them to the requesting process

Deadlock Prevention

- **Circular Wait:** We can impose a total ordering of all resource types and to require that each process requests resources in an increasing order of enumeration.

Deadlock Avoidance

- Deadlock avoidance requires that the operating system be given additional information in advance concerning which resources a process will request and use during its lifetime.
- With this additional knowledge, the operating system can decide for each request whether or not the process should wait.
- **The resource allocation state:** defined by the number of available and allocated resources and the maximum demands of the processes
- A state is safe if the system can allocate resources to each process (up to its maximum) in some order and still avoid a deadlock.
- An unsafe state **may** lead to a deadlock.

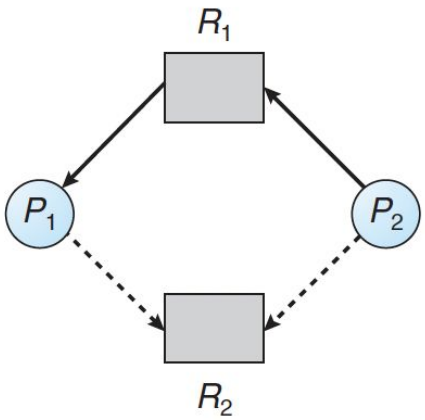
Deadlock Avoidance

- Consider a system with 12 resources and 3 threads: T 0 , T 1 , and T2

	Maximum Needs	Current Needs
T_0	10	5
T_1	4	2
T_2	9	2

Resource-Allocation Graph Algorithm

- If we have a resource-allocation system with only one instance of each resource type, we can use a variant of the resource-allocation graph for deadlock avoidance.
- This graph has claim edges



Banker’s Algorithm

- Used for deadlock avoidance when there are multiple instances of resources
- It uses 4 main data structure
  - Available
  - Max
  - Allocation
  - Need

Banker's Algorithm

Safety Algorithm

1. Let ***Work*** and ***Finish*** be vectors of length m and n, respectively. Initialize ***Work*** = ***Available*** and ***Finish***$[i]$ = ***false*** for $i = 0, 1, \ldots, n - 1$.
2. Find an index i such that both
   a. ***Finish***$[i]$ == ***false***
   b. $\boldsymbol{Need}_i \leq \boldsymbol{Work}$

   If no such i exists, go to step 4.
3. ***Work*** = ***Work*** + $\boldsymbol{Allocation}_i$
   Finish$[i]$ = ***true***
   Go to step 2.
4. If ***Finish***$[i]$ == ***true*** for all i, then the system is in a safe state.

Banker's Algorithm

Resource-Request Algorithm

1. If $\boldsymbol{Request_i} \leq \boldsymbol{Need_i}$, go to step 2. Otherwise, raise an error condition, since the process has exceeded its maximum claim.
2. If $\boldsymbol{Request_i} \leq \boldsymbol{Available}$, go to step 3. Otherwise, P_i must wait, since the resources are not available.
3. Have the system pretend to have allocated the requested resources to process P_i by modifying the state as follows:

$$\boldsymbol{Available} = \boldsymbol{Available} - \boldsymbol{Request_i};$$
$$\boldsymbol{Allocation_i} = \boldsymbol{Allocation_i} + \boldsymbol{Request_i};$$
$$\boldsymbol{Need_i} = \boldsymbol{Need_i} - \boldsymbol{Request_i};$$

Banker's Algorithm

Number of instances of A:10, B:5, C:7

	Allocation	*Max*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

Banker's Algorithm

Number of instances of A:10, B:5, C:7

	Allocation	*Max*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

The sequence P1, P3, P4, P2, P0 satisfies the safety condition

Banker's Algorithm

Number of instances of A:10, B:5, C:7

	Allocation	*Max*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

Now, if P1 requests (1,0,2), will the system remain in safe state?

Banker's Algorithm

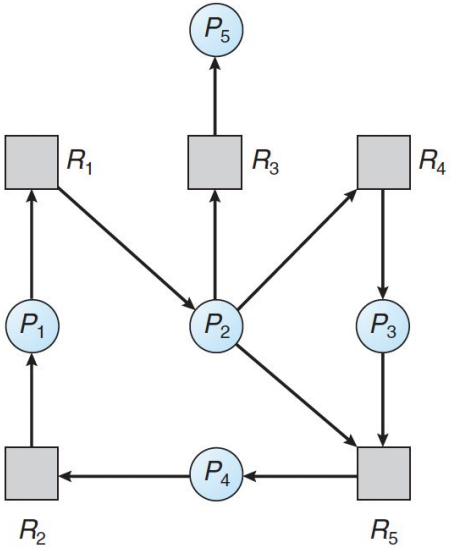
Number of instances of A:10, B:5, C:7

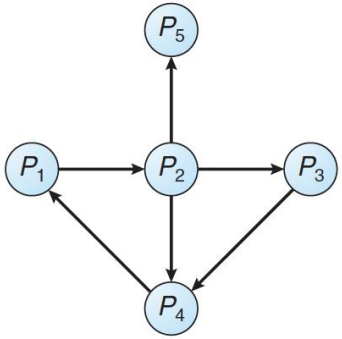
	Allocation	*Max*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

Now, if P1 requests (1,0,2), will the system remain in safe state?
Yes. With the sequence P1, P3, P4, P0, P2

Deadlock Detection

- For deadlock avoidance when there are multiple instances of resources, we can use a variant of the resource-allocation graph, called a wait-for graph





Deadlock Detection

1. Let ***Work*** and ***Finish*** be vectors of length m and n, respectively. Initialize ***Work*** = ***Available.*** For $i = 0, 1, ..., n-1$, if $\boldsymbol{Allocation}_i \neq 0$, then ***Finish***$[i]$ = ***false.*** Otherwise, ***Finish***$[i]$ = ***true.***
2. Find an index i such that both
   a. ***Finish***$[i]$ == ***false***
   b. $\boldsymbol{Request}_i \leq \boldsymbol{Work}$

   If no such i exists, go to step 4.
3. ***Work*** = ***Work*** + $\boldsymbol{Allocation}_i$
   Finish$[i]$ = ***true***
   Go to step 2.
4. If ***Finish***$[i]$ == ***false*** for some i, $0 \leq i < n$, then the system is in a deadlocked state. Moreover, if ***Finish***$[i]$ == ***false,*** then process P_i is deadlocked.

Deadlock Detection

A:7 B:2 C:6

	Allocation	*Request*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

Suppose now that process P2 makes one additional request for an instance of type C.

Recovery from Deadlock

- Process Termination
  - Abort all deadlocked processes
  - Abort one process at a time until the deadlock cycle is eliminated
- Resource Preemption: In this case we need to consider following issues
  - Selecting a victim
  - Rollback.
  - Starvation.